PROGETTAZIONE DI SOFTWARE SICURO

Lezione 10

Aspetti di sicurezza nel ciclo di vita del SW

Elvinia Riccobene

Università degli Studi di Milano - SSRI

Argomenti

- **La sicurezza nel ciclo di vita del software**
- **Testing di Sicurezza**
- **Qualità del software**
- **Attacchi nel software**
- **Ciclo di vulnerabilità del software**
- **Principi di Architetture Sicure**
- **Criteri di certificazione per SW Sicuro**

Sicurezza

Un nuovo aspetto da gestire: la sicurezza

Un nome per due termini:

safety

- il sistema non causa danno all'esterno
- cose cattive non accadono (causate *dal* sistema)

security

- il sistema non subisce danni dall'esterno
- cose cattive non accadono *sul* sistema

Sicurezza: una proprietà relativa

Sicurezza non è una *proprietà assoluta* del sistema

- Il suo significato è *relativo* al contesto di applicazione
- Può essere definita rispondendo alla domanda: sicurezza contro *cosa* e da *chi*?
- Implica stabilire una *policy* di sicurezza

Qualità del software

Safety e security sono proprietà del Sistema.

Possono eseere garantire su SW che ha certe qualità

Qualità del software è la *misura* in cui un prodotto sw soddisfa certe aspettative rispetto sia al suo funzionamento sia alla sua struttura interna

ISO/IEC 9126 definisce la qualità del software secondo tre differenti viste:

- Qualità esterne (black box view)
- Qualità interne (white box view)
- Qualità in uso

Qualità del software

qualità esterne

- Correttezza
- Affidabilità
- Usabilità
- Portabilità
- Interoperabilità
- Robustezza

qualità interne

- Riusabilità
- Verificabilità
- Facilità di manutenzione

qualità d'uso

- Efficienza
- Portabilità
- Interoperabilità

Qualità esterne del SW per la safety

Correttezza (o funzionalità): un SW è corretto se rispetta le specifiche *funzionali* di progetto

- Richiede la specifica

Affidabilità (o reliability): un SW è affidabile se si comporta "*come previsto dalla (spec. ei) requisiti*"

  - Valutata come la "*probabilità di assenza di fallimenti in un dato intervallo di tempo*"
  - SW corretto è affidabile, ma non vale il viceversa

Robustezza: un SW è robusto se si comporta in modo ragionevole anche in circostanze non previste dalle specifiche di progetto

- es. input incorretti, rotture di dischi, ecc.)

Un sw affidabile e robusto *può* essere *safe*

Altre qualità connesse all'usabilità

Usabilità: facilità d'uso da parte dell'utente

- Qualità difficile da valutare; molto soggettiva
- Comporta la definizione di *expected user* e richiede *test di usabilità*
- Influisce molto sulle interfacce utente

Portabilità: un SW è portabile se può funzionare su più piattaforme

- Esempio: programmi in Java

Interoperabilità: abilità di un sistema di integrarsi e cooperare con altri sistemi

Altre qualità connesse all'usabilità

Efficienza:

- un SW è efficiente se usa intelligentemente le risorse di calcolo (ad es. memoria, tempo di processamento, metodi di comunicazione)
  - Può essere valutata attraverso analisi di complessità o la simulazione di scenari critici
  - Può essere influenzata dalla scalabilità:
    - una soluzione che funziona in piccolo può non funzionare in grande
  - Può cambiare con la tecnologia
  - Può influenzare la facilità d'uso

Qualità interne del SW (1)

Riusabilità:

- un SW è riusabile se può essere usato, in tutto o in parte, per costruire nuovi sistemi

Verificabilità:

- possibilità di *dimostrare* a posteriori la correttezza o altre caratteristiche del software
  - Verifica formale
  - Testing
  - Verifica a runtime

Qualità interne del SW (2)

Facilità di manutenzione:

- facilità nel realizzare adattamenti o evoluzioni; agio nel correggere gli errori

Un SW è facile da manutenere se:

- è strutturato in modo tale da facilitare la ricerca degli errori (*modifiche correttive*)
- la sua struttura permette di aggiungere nuove funzionalità al sistema (*modifiche perfettive*)
- la sua struttura permette di adattarlo ai cambiamenti del dominio applicativo (*modifiche adattative*)

Qualità del SW sicuro

Oltre le qualità comuni a tutto il SW, un SW sicuro deve godere delle seguenti qualità (*security goals*) :

- prevenzione
- tracciabilità e controllo (o *auditing*)
- monitoraggio
- privatezza e confidenzialità
- sicurezza a diversi livelli
- anonimato
- autenticazione
- integrità

Qualità del SW sicuro

Prevenzione

- Anticipare possibili attacchi
  - A livello di progettazione
  - A livello di implementazione
  - A livello d'uso
- Richiede l'analisi del rischio

Tracciabilità

- Meccanismo che consente di stabilire in modo inequivocabile la relazione di *causa/effetto* tra elementi, eventi, o processi
  - Utile per riparare ad un attacco

Qualità del SW sicuro

Controllo (o *auditing*)

- Processo di controllo di un sistema, effettuato sulla base del *confronto* tra le *attività* svolte sul sistema con le *politiche* (si sicurezza) stabilite al fine di determinare la loro conformità.
  - Utile per suggere migliorie sulla politica
  - Non è una tecnica di prevenzione, ma è utile per dissuadere da potenziali attacchi

Monitoraggio

- È auditing in tempo reale
- È possibile monitorare a diversi livelli
  - Approcci semplici: controllo di segnature note che identificano un attacco in corso
  - Approcci complessi: uso di *asserzioni* nel codice

Qualità del SW sicuro

Privatezza

- E' il diritto di un individuo di stabilire se, come, quando e a chi l'informazione che lo riguarda può essere rilasciata

Confidenzialità

- Assicura che certi servizi e informazioni siano accessibili solo ad utenti autorizzati

Integrità

- Nel contesto della sicurezza, integrità va intesa come "*rimanere lo stesso*", ossia non modificato
  - le informazioni non devono essere modificate o cancellate in modo non autorizzato o *improprio*

Qualità del SW sicuro

Autenticazione

- Va intesa come la proprietà di conoscere l'identità di *chi* accede ad un servizio
  - E' una proprietà critica per la sicurezza e un SW sicuro include sempre elementi di autenticazione (*principio di responsabilità*)

Anonimato

- Va intesa come la proprietà di mantenere segreta (o non accessibile pubblicamente) l'origine di certi dati
  - E' una proprietà con doppio risvolto
  - Dati potrebbero essere usati in cattivo modo? E come?

Qualità del SW sicuro

Sicurezza a più livelli

- Alcuni tipi di dati e informazioni sono più segrete di altre
  - Informazioni classificate in: pubbliche, confidenziali, top-secret, ecc.

- È necessario disporre di sistemi in grado di gestire la segretezza di dati e informazioni a più livelli di competenza
  - principio di responsabilità

SICUREZZA E PROCESSO DI SVILUPPO

Modello ottimale per SW sicuro

- Il modello a spirale è il modello di sviluppo ottimale per poter effettuare un’ efficace analisi di sicurezza
  - Prevede al suo interno la fase di analisi dei rischi
  - E' un modello di processo flessibile
  - Consente il raffinamento successivo
  - Consente la validazione già nelle fasi iniziali con possibilità di integrare soluzioni trovate a posteriori

Politiche e Meccanismi di Sicurezza

Politica dice cosa è e cosa non è consentito

- Una politica definisce la "security" di un sistema
- Spesso definita in linguaggio naturale ed in modo ambiguo

Meccanismo di sicurezza è un metodo (**tecnico** o **procedurale**) strumento per applicare e garantire una politica di sicurezza

Composizione di politiche

- In caso di composizione di sistemi sicuri
- Se le politiche sono in conflitto, le discrepanze possono causare vulnerabilità per la sicurezza dell'intero sistema

Sicurezza vs funzionalità

- Progettare un sistema SW che garantisca sia le proprietà di sicurezza che le qualità del software è un' attività complessa

- L' analisi di sicurezza (detta anche *analisi e gestione dei rischi*) è importante per riuscire a bilanciare tra sicurezza e funzionalità
  - È indispensabile per decidere quali obbietti realizzare e quali proprietà e qualità garantire

- L' analisi di sicurezza, è un processo **ortogonale** al processo di sviluppo del SW

Analisi dei rischi (1/2)

- Ha l'obiettivo di identificare i rischi possibili e di valutare strategie per prevenirli o affrontarli
  - L'analisi dei rischi influisce sul processo di auditing

- I rischi vanno classificati in base alla loro severità
  - Non è una classificazione assoluta ma context-sensitive
  - La classificazione è importante per allocare risorse per il testing e l'analisi del sistema

Analisi dei rischi (2/2)

- È difficile comparare sistemi in termini di sicurezza
  - Non esiste una metrica "assoluta" ma linee guida molto generali per la valutazione e la comparazione di sistemi sicuri
    - Come eseguire l'analisi di sicurezza
    - Che tipo di rischi considerare
- Il **livello di protezione** è *funzione della probabilità che un attacco si verifichi e degli effetti dell'attacco qualora succeda*
  - Se un attacco è improbabile, il livello di protezione è basso
  - Se un attacco improbabile causerebbe lunghi ritardi nella produzione di una società, ed uno probabile sarebbe solo un fastidio, sforzo maggiore va messo nel prevenire l' attacco improbabile

Esempi di rischio per la sicurezza

Un attacker non autorizzato ottiene l'accesso al sistema usando credenziali autorizzate.
Una persona autorizzata accede a risorse che dovrebbero essergli proibite.
Il sistema di autenticazione non riesce a identificare un attacker non autorizzato.
Un attacker ottiene l'accesso al database grazie a un attacco SQL poisoning.
Gestione non corretta di sessioni HTTP.
I cookie di una sessione HTTP sono rivelati a un attacker.
Dati confidenziali non cifrati.
Le chiavi di cifratura trapelano a potenziali attacker.

Scenari da prevenire

Pianificare possibili soluzioni per il rischio 1:

Un utente malintenzionato non autorizzato ottiene l'accesso a un sistema utilizzando credenziali autorizzate

Possibili scenari da prevenire:

- L'utente ha impostato password deboli che possono essere indovinate da un utente malintenzionato.
- Il file delle password di sistema è stato rubato e le password sono state scoperte dall'aggressore.
- L'utente non ha configurato l'autenticazione a due fattori.
- Un utente malintenzionato ha scoperto le credenziali di un utente legittimo tramite tecniche di ingegneria sociale.

Sicurezza e requisiti (1)

- Solitamente i **requisiti di sicurezza** sono categoria a parte rispetto ai requisiti non funzionali
  - Spesso parte dei reqs di performance
- Nella definizione dei requisiti connessi alla sicurezza occorre identificare:
  - **ciò** che va protetto, **da chi** va protetto e per **quanto tempo** va protetto
  - **quanto** vale mantenere certi dati protetti
- I requisiti devono essere chiari e completi
  - Es. "Questa applicazione deve usare la crittografia quando necessario"
    - Requisito mal posto perché descrive la soluzione senza diagnosticare il problema

Sicurezza e requisiti (2)

- Il documento dei requisiti deve specificare *cosa* un sistema deve e non deve fare, ma anche *perchè* (o *Rationale*) deve farlo
  - Es. "I numeri delle carte di credito vanno protette contro potenziali furti perchè sono informazioni delicate"
    - Requisito ben posto; la scelta di come proteggere queste informazioni può essere rimandata alla fase di specifica

Sicurezza e specifica

- La specifica deve integrare eventuali possibili soluzioni a rischi individuati

- L'analisi di sicurezza continua in fase di specifica
  - Aiuta avere una specifica:
    - **dettagliata** che descriva la reazione del sistema a circostanze critiche
    - **formale**, completa e non ambigua, ma anche chiara e comprensibile
    - **eseguibile** per avere un feed-back immediato

- Se nuovi rischi sono individuati, vanno riflessi nei requisiti

Sicurezza e design

- Nella fase di design occorre attuare il *security plan*
- Va controllato/garantito:
  - come i dati passano da una componente all'altra e garantirne l'integrità
  - utenti, ruoli e diritti che sono esplicitamente dichiarati o implicitamente supposti dal design (sulla base di politica per confidenzialità e privacy)
  - ogni soluzione potenzialmente applicabile a problemi individuati per prevenire attacchi noti
  - la relazione di *trust* di ciascuna componente

Sicurezza ed implementazione

- L' implementazione è una fase critica
  - Molti attacchi sono possibili a causa di errori introdotti in fase di stesura di codice

- Compito dell' ingegnere della sicurezza è quello di:
  - identificare le linee guida per la stesura del codice
  - applicare (se possibile) metodi formali per la prova della correttezza del SW
  - predisporre tecniche per il controllo del codice (*code audit*)

Sicurezza e testing

Per sistemi sicuri si distingue tra:

- **testing funzionale**: comporta il mettere alla prova un sistema per determinare se il sistema fa ciò che si suppone debba fare in circostanze normali o critiche
- **testing di sicurezza**: mira a trovare vulnerabilità che possono essere sfruttate da un utente malevolo e a fornire prove convincenti che il sistema sia sufficientemente sicuro.
  - Code coverage rimane ancora un metodo importante per il testing di sicurezza
    - Permette di scoprire se esistono parti di codice che non vengono mai eseguite; codice non eseguito può contenere minacce per la sicurezza

Test di sicurezza

- I test dovrebbero dimostrare che il sistema può resistere agli
  - attacchi individuati in fase di analisi del rischio,
  - attacchi che tentano di iniettare malware e
  - attacchi che tentano di corrompere o sottrarre dati e identità degli utenti.
- Test di sicurezza completi richiedono una conoscenza specialistica delle vulnerabilità del software e approcci ai test in grado di individuare tali vulnerabilità.
  - Ad esempio, per il rischio 1 visto prima:
    - eseguire un test per verificare che il codice che consente agli utenti di impostare le proprie password controlli sempre la robustezza delle password.

PRINCIPI DI ARCHITETTURE SICURE

Architettura per la sicurezza

Architettura per la sicurezza:
insieme dei principi d'alto livello e delle decisioni per la progettazione di sistemi da ritenersi sicuri

É un framework concettuale per la progettazione sicura

- Secondo i 4 principi: proteggi, scoraggia, scopri e reagisci (protect, deter, detect, e react)

Bilancia tra

- troppa sicurezza
- controlli superficiali per la sicurezza

Architettura vs Politica

Un'architettura per la sicurezza è diversa dalla politica di sicurezza (*policy*)

- Una politica di sicurezza stabilisce:
  - le **regole** su chi può avere accesso ed a che tipo di dati
  - le **procedure di testing** necessarie a certificare che l'applicazione può girare in rete in modo sicuro
- Una politica di sicurezza va stabilita a priori ed usata come **linea guida** per definire la architettura dell'applicazione

Principi di un'architettura sicura (1)

Ecco una lista di principi base per definire architetture sicure:

- inizia facendo domande e decidi *che livello di sicurezza* è *sufficiente*
- progetta con in mente il nemico
- conosci e rispetta la *chain of trust*
- definisci privilegi adeguati
- testa le azioni possibili contro la *policy*

Principi di un'architettura sicura (2)

- mantieni in memoria lo stato minimale
- garantisci un adeguato livello di *fault tolerance*
- pianifica l' *error-handling*
- applica il principio di *graceful degradation*
- garantisci che un fallimento lasci il sistema in una configurazione sicura
- applica misure di sicurezza adeguate all'utente
- suddividi la progettazione in moduli
- garantisci la possibilità di ricostruire gli eventi

Progettare con *in mente il nemico*

- *Adversary principle*: progetta il SW come se il nemico più astuto lo attaccherà

- Il progettista dovrebbe anticipare gli attacchi di "avversari intelligenti, razionali e irrazionali" (GASSP) e prevedere possibili soluzioni
  - GASSP: Generally Accepted System Security Principles

- E' impossibile progettare con *in mente il nemico* senza un realistico senso di cosa il nemico potrebbe voler attaccare del SW

Conoscere e rispettare la *chain of trust*

- Non invocare programmi non fidati da programmi fidati
  - Spesso violato quando un'applicazione invoca un comando di sistema per eseguire un'operazione
    - Es. Vulnerabilità della utility *man* di Unix (UNSAFE string) che esegue con pieni privilegi su vecchie versioni di UNIX
- **Regola generale**: un programma NON deve delegare l' *autorità* di fare un'azione senza delegare anche la *responsabilità* di controllare se l'azione sia appropriata
- Un'applicazione rispetta la chain of trust se:
  - valida tutto quello che riceve in input
  - non passa tasks ad entità meno fidate
  - fornisce servizi sicuri (certificati)

Privilegi adeguati

- Un’applicazione necessita di adeguati privilegi e accessi per operare

- Seguire il principio del minimo privilegio
  - Per leggere un record non aprire il file con accesso anche in scrittura
  - Per creare un file in una directory condivisa, fa uso del "group access“ o delle liste di controllo d’accesso

Test delle azioni contro la *policy*

- Principio di mediazione completa: passo per passo, occorre testare ogni azione che è possibile tentare contro la policy

- La mediazione completa è necessaria a garantire che le decisioni *step-by-step* prese in un dato momento dal SW sono conformi alla politica di sicurezza del sistema

Offuscamento delle informazioni

- Principio del non offuscamento: nascondere la modalità di funzionamento di una componente SW o il registro in cui è memorizzato un particolare parametro di policy è pericoloso
  - **"Security through obscurity doesn't work!"**

- Dibattito su *sw open source* per aumentare la sicurezza

Mantenere lo stato minimale

- **Stato** = informazione che un programma deve tenere in memoria durante un transazione o esecuzione di un comando

- Principio del *minimal retained state*: un programma deve tenere in memoria lo stato minimale
  - Contresempi sono SYN flood attacks o CGI scripts attacks, buffer overflow

Livello di *fault tolerance*

Per avere certificazione CERT, un apparato deve

- Identificare le funzionalità critiche
  - Uso delle tre R:

    Resistance: la capacità di impedire un attacco

    Recognition: la capacità di riconosce attacchi e la misura del danno

    Recovery: la capacità di fornire servizi ed assetti essenziali durante un attacco e ripristinare tutti I servizi dopo l'attacco
- Valutare il ruolo del SW nel contesto della *fault tolerance and continuity planning* della compagnia
  - Esempio di sw che accede a DB

Graceful degradation

- Principio di *graceful degradation*: quando si verifica un guasto, il sistema non si ferma ma continua ad operare in modo ristretto o con funzionalità ridotta (*degrated*)

- Esempio di design senza graceful degradation
  - Packets del protocollo TCP

- Esempio di design con graceful degradation
  - Gestione della memoria nel sistema operativo VMS della Digital

Fallimento in modo sicuro

- Principio di *fault safely*: in caso di fallimento, un sistema deve terminare in una configurazione sicura
  - Se il server su cui è il farewall muore, la rete deve essere lasciata chiusa
  - Se il programma che controlla il time lock di una cassaforte fallisce, la cassaforte deve rimanere chiusa
  - Se fallisce il programma di controllo di una porta a chiusura elettronica, la porta deve rimanere aperta
  - Esempio più complesso: fallimento del programma che implementa il criterio di “morte celebrale” in un dispositivo medico

Misure di sicurezze adeguate all'utente

- Principio della *accettabilità psicologica*: è importante usare modelli mentali e paradigmi tratti dal mondo reale
  - Esempio: seleziona un'interfaccia utente che renda facile fare le cose corrette

- Se le misure di sicurezza di un sistema sono così onerose ed irritanti che l'utente le evita, la sicurezza del sistema è compromessa

Responsabilità individuali

- Principio di *accountable individuals*: una architettura di successo deve garantire che gli individui siano responsabili delle proprie azioni

- Questo principio richiede che:
  - ogni utente abbia ed usi un account personale
  - sia difficile per un utente spacciarsi per un altro
  - deve essere chiaramente stabilita la responsabilità della sicurezza delle risorse coinvolte

Ricostruzione di eventi

- Principio di *auditability*: deve essere possibile ricostruire la sequenza di eventi che hanno condotto a certe azioni chiave (es. cambio di dati)

- Questo principio richiede che l'applicazione e il sistema di host creino e mantengano gli *audit log*

Modularizzazione e semplicità

- Principio di modularizzazione:
  - suddividere la progettazione in moduli
  - definire con precisione i punti di interfaccia tra moduli (argomenti, strutture di memoria comuni, ecc.)
  - limitare privilegi ed risorse ai moduli che realmente ne necessitano

- Funzioni che richiedono privilegi speciali devono essere isolate e codificate in moduli/componenti separati
  - principi di alta coesione e basso accoppiamento
- Realizzare sistemi che siano il più semplici possibili

TCP SYN flood

- Dall' analisi degli attacchi SYN flood del protocollo TCP, i seguenti principi di progettazione sono stati violati:
  - progettare con il mente il nemico
  - garantire un adeguato livello di *fault tolerance*
  - il principio di *graceful degradation*
  - controllo del consumo di risorse
  - principio dello stato minimale

THREAT, ATTACCHI E CICLO DI VULNERABILITÀ

Cosa è un attacco

Un attacco ad un sistema è ogni atto malevolo contro un sistema o un complesso di sistemi

Nell'ambito di un attacco vanno distinti:

- *goal*
- *sottogoal*
- *attività*
- *eventi*
- *conseguenze*
- *impatto*

Fasi di un attacco (1)

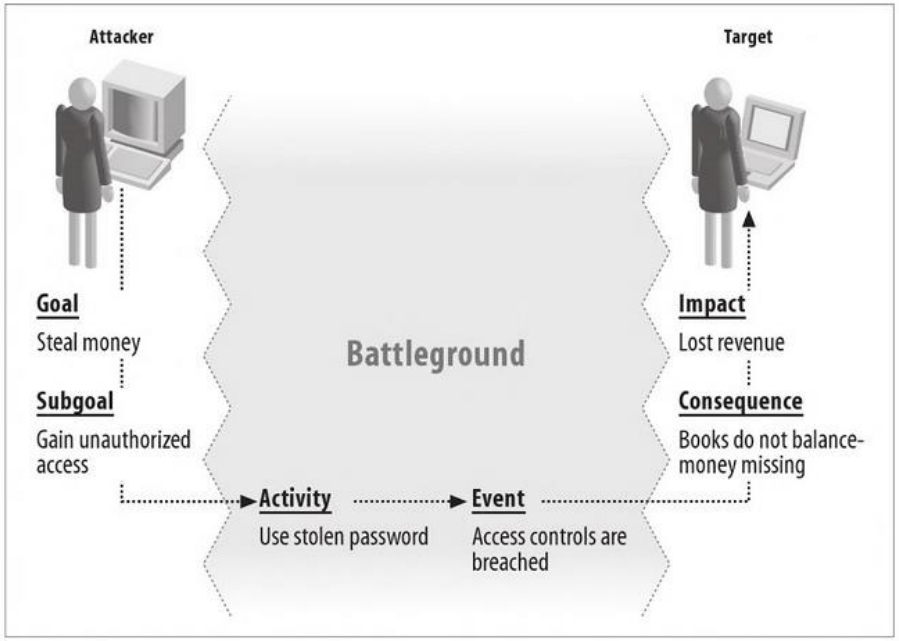
- *Goal*
  - Il danno causato al sistema
    - Es. rubare denaro, servizi, ecc.
- *Sottogoal*
  - Gli obbiettivi intermedi necessari a raggiungere il goal
    - ES. ottenere privilegi o autorizzazioni dal sistema
- *Attività*
  - Le attività che un attaccante deve svolgere per raggiungere uno o più sottogoal
    - Es. usare credenzialità (es. username e password) rubate per logarsi al sistema; spacciarsi per un diverso computer, o user, o device; invadere la rete con pacchetti malformati; ecc.

Fasi di un attacco (2)

- *Eventi*
  - L'occorrenza di attacchi causati dalle attività
    - Es. ottenere un accesso improprio; sospendere il processamento di una richiesta; esaurire lo spazio di memoria; fermare un sistema o un programma
- *Conseguenze*
  - Le conseguenze del verificarsi di un evento
    - Es. incorrettezza dei estatti conto bancari; non disponibilità di un sistema a processare richieste di business; ecc.
- *Impatto*
  - Effetti di business
    - Es. offuscare la reputazione di una compagnia; perdita di guadagno; ecc.

Esempio di attacco

Attacker



Target

- *Goal:*
  - rubare denaro
- *Sottogoal:*
  - ottenere accesso non autorizzato
- *Attività:*
  - usare password rubata
- *Eventi:*
  - controlli d'accesso violati
- *Conseguenze:*
  - mancanza di bilancio
- *Impatto:*
  - perdita di guadagno

Modalità di attacco

Come un attacker attacca un sistema?

Il *come* dipende spesso dal *perché* e dalla *competenza* in materia di sicurezza

Per **rafforzare** la sicurezza di un'applicazione è importante domandarsi

Cosa può accadere e

Come può accadere

Tipi di attacco

In fase di

- *Progettazione*

  Mentre si sta progettando l' applicazione

- *Implementazione*

  Mentre si sta scrivendo l' applicazione

- *Operazione*

  Dopo che l' applicazione è in produzione

Possibili attacchi in fase di progettazione

I più comuni attacchi in fase di progettazione

- Man-in-the-middle attack
  - Replay attack
- Session hijacking attack
- Session killing attack
- Race condition attack (o TOCTTOU)
- Sniffer attack

Altri possibili attacchi

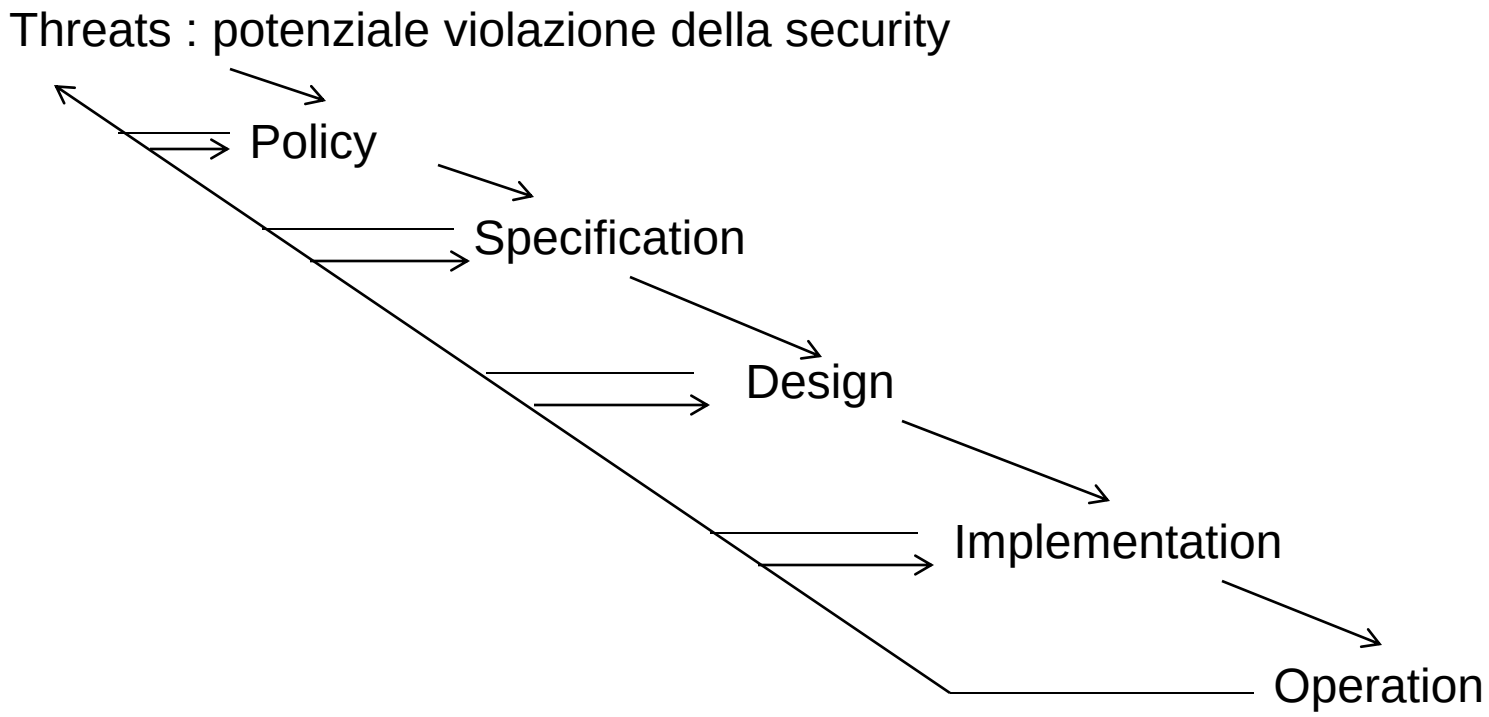
Comuni attacchi in fase di implementazione

- Buffer overflow
- Back door
- Parsing error

Comuni attacchi in fase di operazione

- Denial-of-service
- Default accounts
- Password cracking

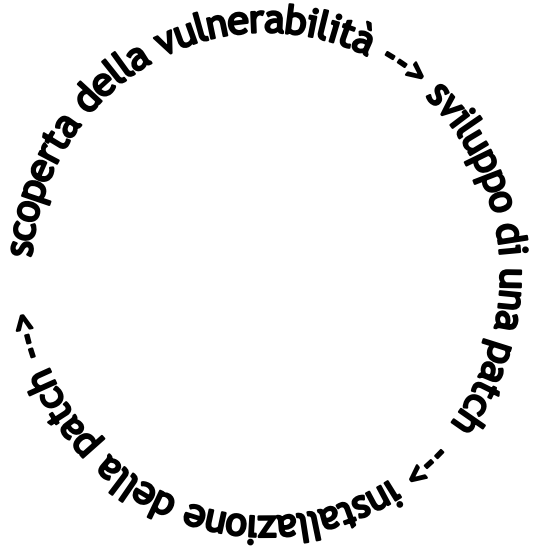
Processo di gestione delle minacce



Ciclo di vulnerabilità

- E' una sequenza di eventi che si susseguono molto comunemente nel mondo della sicurezza
  - Va dalla determinazione alla eliminazione di una vulnerabilità del SW

- E' un ciclo **ortogonale** al ciclo di vita del SW
  - Solitamente ha luogo dopo la distribuzione del SW



Ciclo di vulnerabilità: sequenza di eventi

1. Viene scoperta una nuova vulnerabilità in una componente SW
2. Si cerca una soluzione al problema:
   - Si **analizza** la vulnerabilità, **sviluppa** una soluzione e la si **testa** in un ambiente controllato, si **distribuisce** la soluzione
   - Analisi interna o affidata ad ente esterno
3. La patch distribuita (spesso rilasciata come parte di update automatico), viene installata dagli addetti
4. Tecnici della sicurezza analizzano frammenti di codice per scoprire vulnerabilità simili

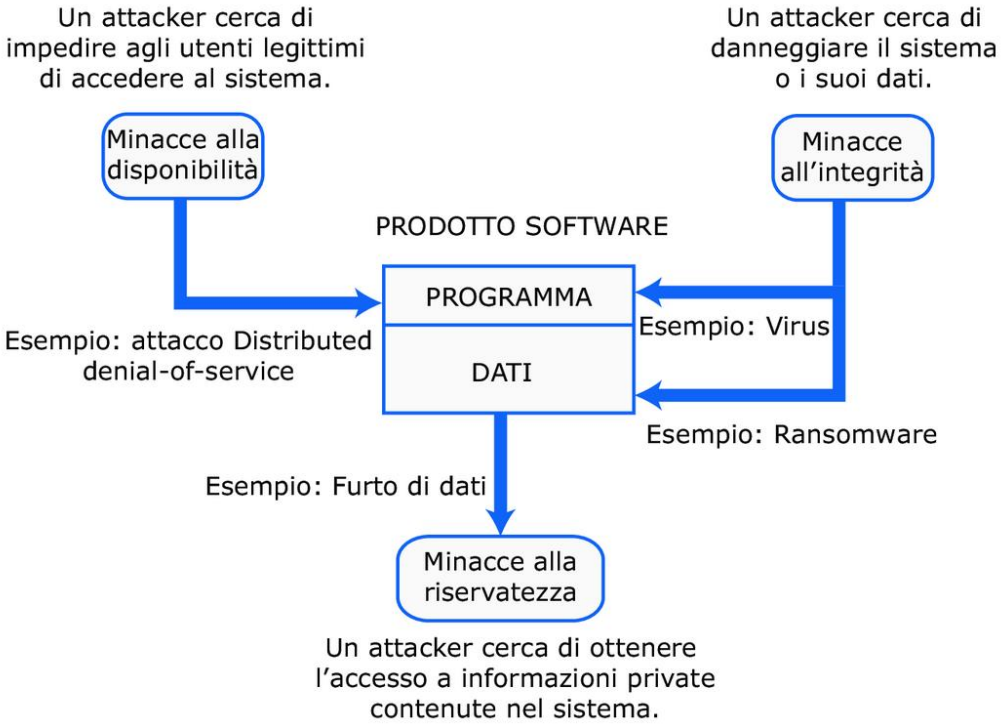
A questo punto il ciclo si ripete !!!

Ciclo di vulnerabilità: conseguenze

- Sistemi e reti vulnerabili raramente vengono riparate durante il ciclo di vita della vulnerabilità
  - Le patch sono ricevute come parte di un versione aggiornata del SW
- Se la vulnerabilità è seria, o gli attacchi sono gravi, anche i media ne danno pubblica informazione, spesso con catastrofiche conseguenze
- Se non si agisce in modo tempestivo, utenti *malevoli* possono riuscire a sfruttare la vulnerabilità causando e propagando danni senza misura

Ciclo di vulnerabilità: quanto frequente?

- Per avere un'idea delle dimensioni del problema:
  - su 100 applicazioni Internet che forniscono servizi critici, esistono circa 10 mila bug di sicurezza;
- Principali tipologie di minacce:



CRITERI DI VALUTAZIONE PER SISTEMI SICURI

Criteri di valutazione della sicurezza

- La valutazione di sistemi critici sicuri richiede la definizione di uno standard per l'analisi
- Nel 1985 viene pubblicato l' **Orange Book**
  - *Department of Defence Trusted Computer Systems Evaluation Criteria*
  - In collaborazione tra il NSA (National Security Agency) ed il DoD (Department of Defence)
  - Noto come: TCSEC - Trusted Computer System Evaluation Criteria
- Successivamente sono stati introdotti
  - Information Technology Security Evaluation Criteria (ITSEC) – TCSEC a livello europeo
  - Federal Criteria
  - Common Criteria

Criteri di valutazione della sicurezza

Obiettivo di questi standard:

- Definire criteri e scale di valori per ***misurare*** l'affidabilità di prodotti e di sistemi
- Definire complesso di ***documenti*** ed artifatti in base ai quali fare la valutazione
- Definire una ***metodologia*** di valutazione da seguire

Finalità degli standard:

- Sensibilizzare sull'aspetto security oltre che safety
- Poter confrontare prodotti diversi
- Stabilire regole di valore internazionale

Valutazione di prodotti e di sistemi

- La valutazione di **prodotti** (entità utilizzabili in diverse situazioni e diversi ambienti e con diversi gradi di sicurezza)
  - Devono soddisfare requisiti delle *security class* dell' **orange book** e *protection file* dei **Common Criteria**
- La valutazione di **sistemi** (insieme di vari prodotti assemblati)
  - Vengono valutate le singole componenti come specificato dall' **ITSEC**
- La valutazione è affidata ad un attore indipendente dal costruttore
  - Stati Uniti: agenzia governativa
  - Europa: agenzie certificate ad assolvere il compito

Classi di valutazione (orange book)

Parametri di valutazione:

- ***Politiche di sicurezza*** (mandatorie o discrezionali)
- ***Etichettatura degli oggetti*** (in base al livello di criticità)
- ***Identificazione dei soggetti*** (garantita mediante meccanismi di autenticazione)
- ***Auditing*** (accessi continuamente monitorati)
- ***Documentazione*** (importante per uso e testing)
- ***Affidabilità*** (meccanismi di sicurezza attivi per evitare il blocco dei sistemi)

Classi di valutazione (TCSEC)

4 classi di ripartizione (gerarchiche):

- D (*minimal protection*): no classi, no meccanismi di protezione
- C (*discretionary protection*): controllo di sicurezza discrezionale (eventualmente autenticazione e auditing) e politica per il riuso (C1 -> C2)
- B (*mandatory protection*): controllo obbligatorio
  - B1 (labelled security protection): implementano politiche multilivello secondo il modello di Bell-La Padula
  - B2 (structured protected): ulteriori vincoli in fase di progettazione per la protezione della memoria ed esecuzione di processi in spazi limitati
  - B3 (security domains): meccanismi di prevenzione di attacchi esterni e sistemi di ripristino dopo anomalie

Classi di valutazione (TCSEC)

4 classi di ripartizione (gerarchiche):

- …..
- A (*verified protection*): uso di metodi formali per la verifica di sicurezza
  - Bisogna verificare formalmente che TCB siano corrette rispetto al modello formale delle politiche e che implementano le politiche prefissate
    - TCB (trusted computing base): componenti HW e Sw *fidati*

Common Criteria ed Evaluation Methodology

- **Common Criteria** vers. 2 (~ 600 pagine), standard ISO 15408
  - non fornisce una lista di requisiti di sicurezza o funzionalità che il sistema deve possedere
  - descrive solo un framework concettuale da seguire per lo sviluppo di un sistema (dai reqs alla codifica)
  - definisce *profili di protezione* per ogni tipo di prodotto IT, incluso hardware, firmware, software

- **Common Evaluation Methodology** (circa 400 pagine) definisce le modalità di valutazione di un sistema in base ai criteri comuni

Processo di valutazione: esempio (1)

1. Wall Street produce un profilo di protezione per il firewall usato per proteggere le macchine che contengono informazioni finanziarie dei clienti.
2. Il profilo viene valutato in base alle Common Evaluation Methodology per garantire che esso sia consistente e completo.
3. Ottenuta la valutazione, il profilo viene pubblicato (target di sicurezza).
4. Un vendor realizza una sua versione (target di valutazione) di firewall dotato del profilo di protezione definito da Wall Steet
5. Il *target di valutazione* viene inviato ad un istituto accreditato per la valutazione rispetto al *target di sicurezza.*

Processo di valutazione: esempio (2)

6. L'istituto applica la Common Evaluation Methodology per determinare in base ai common criteria se il target di valutazione soddisfa il target di sicurezza.
7. Se la valutazione ha esito positivo, la documentazione di testing dell'istituto viene inviata al NVLAP (National Voluntary Lavoratory Accreditation Program) per controllarne la correttezza.
8. Se gli atti vengono approvati, il prodotto ottiene la **certificazione** di prodotto valutato in base ai common criteria.

Limiti dei *common criteria*

- La valutazione della sicurezza è un'attività molto più complessa dell’ applicare un target di sicurezza ad un target di valutazione

- I problemi di sicurezza sono *context-sensitive* ed è difficile valutarli in base a criteri comuni

- Come molti standard, definiscono il "cosa" e non il "come" mentre nel campo della sicurezza è fondamentale sapere *come* affrontare problemi di sicurezza e gestire i rischi

Valutazione delle smart cards

- Esempio di dominio applicativo che ha customizzazione i common criteria.

- Dal 2001 lo Smart Card Security User's Group sta cercando di definire dei Common Criteria per la valutazione delle smart cards.
  - Esistono due profili per la sicurezza
    a) Quello europeo che mira maggiormente alla protezione fisica dei dati
    b) Quello americano che mira maggiormente alla protezione SW e alla prevenzione di attacchi noti